Project Statement: Financial Ledger & Digital Wallet API

The Problem: The Vanishing Money Glitch

In poorly designed databases, simple balance updates can easily overwrite each other during simultaneous network requests, or a single network retry can accidentally trigger a duplicate charge. When dealing with user finances, missing funds or double charges are unacceptable. A backend system is required that treats money as an absolute, verifiable truth rather than just a number in a database that can be casually overwritten.

The Solution: A Mathematically Sound Backend

This project builds the backend infrastructure for a digital wallet backed by a strict, tamper-evident, double-entry ledger. By enforcing strict mathematical rules and using clean API design, the system guarantees that every single credit has an equal and opposite debit. This ensures that no matter how many times a bad network connection retries a transfer, a user is never double-charged, and no matter how many concurrent requests hit the system, balances remain perfectly accurate.

1. Project Description

Build the backend infrastructure for a digital wallet and financial ledger system. The system shall expose a RESTful API to create wallets, record transfers between accounts, and maintain a mathematically sound, tamper-evident double-entry ledger. The focus of this project is core computer science principles—ACID transactions, data integrity, idempotency, and clean API design.

2. Functional Requirements

- **FR-1 (Wallet Initialization):** Create user accounts with distinct currency wallets. Every balance is backed by double-entry ledger entries (every credit must equal a debit).
- **FR-2 (Idempotent Transfers):** Transfer funds between two wallets. The endpoint must require an idempotency key (header) to ensure retried network requests never double-charge an account.
- **FR-3 (Concurrent Balance Mutability):** Safely handle multiple simultaneous incoming/outgoing transfer requests on the same wallet without data race conditions or balance corruption.
- **FR-4 (Immutable Audit Trail):** Ledger records cannot be updated or deleted once written. Any corrections or cancellations must be recorded as an explicit reversal transaction.

- **FR-5 (Ledger Reconciliation):** An administrative endpoint that scans ledger entries and verifies that the sum of credits equals debits, and that current wallet balances match the calculated ledger sums.

3. Product-Specific Agent Skills

- **simulate-transfer:** Run a complete product workflow and verify ledger consistency: 1. Create two test wallets. 2. Record initial balances. 3. Execute a transfer with an idempotency key. 4. Fire two concurrent transfer requests to test race prevention. 5. Call the reconciliation endpoint and summarize final balances.

4. Product-Specific Testing Requirements

- **TQ-1:** Unit tests for accounting calculations, ledger balance assertions, and idempotency key hashing.
- **TQ-2:** Integration tests verifying database transactions and database rollback behavior on simulated transfer failures.
- **TQ-3:** System (end-to-end) tests for each of FR-1 to FR-5.
- **TQ-4:** Negative tests: negative balance attempts, insufficient funds, invalid idempotency payloads, malformed IDs, and simultaneous concurrent transfers (proving race safety).
- **TQ-5:** CI tests shall run against an isolated test database (e.g., an in-memory or fresh local database instance) on every push.
- **TQ-6:** The system shall prevent floating-point rounding errors. Test that monetary amounts are strictly handled using integer minor units (e.g., cents/paise) or exact decimal types.
- **Security Testing:** Input validation (all API parameters), Negative amount prevention (preventing users from sending negative values to inflate balances).

5. Database Architecture: Keeping It Simple

- **Zero Configuration:** To avoid operational overhead, teams are strongly encouraged to use SQLite. It runs serverless as a single file on disk (or in-memory for unit tests).
- **ACID by Default:** SQLite provides immediate support for transactions (BEGIN, COMMIT, ROLLBACK), which are required for double-entry bookkeeping.
- **Handling Concurrency:** Teams can enable SQLite's WAL mode (PRAGMA journal_mode=WAL;) or use simple database transaction locks to easily satisfy FR-3 without needing distributed lock servers.